Brute-Force Login Detection

Technical Investigation Report

Dataset: tagged_logs.txt

Tool: Splunk

Date: April 2026

BY:

Muhammad Ashar Latif

Introduction:

Brute-force login attacks represent one of the most damaging and persistent threats in modern digital landscape. In a brute-force attack, an attacker uses automated tools to systematically attempt large number of username and password combinations against an authentication service until access is gained.

Unlike sophisticated exploits that target software vulnerabilities, brute-force attacks require no specialized knowledge of the target system, making this such that even low-skill attackers can remain highly effective against system with weak or default credentials.

There are three primary variants of this attack. In classic brute force, a single account is targeted with every possible password combination until the correct one is found. In password spraying, one common password is tried against many accounts simultaneously, evading per-account lockout policies. In credential stuffing, previously breached username-password pairs from other services are replayed against new targets. All three variants appear in SSH authentication logs and can be detected through the same fundamental indicator - an abnormally high volume of failed authentication events from one or more source IP addresses.

From a Security Operations Center (SOC) perspective, brute-force detection is a foundational capability. MITRE ATT&CK classifies this family of techniques under T1110 - Brute Force, with sub-techniques covering dictionary attacks (T1110.001), password cracking (T1110.002), password spraying (T1110.003), and credential stuffing (T1110.004). Early detection is critical - a successful brute-force breach grants the attacker an authenticated foothold from which lateral movement, privilege escalation and data exfiltration becomes significantly easier to execute.

This report documents a complete brute-force detection investigation conducted using Splunk against LogHub SSH authentication log dataset. This report covers data ingestion, field extraction, 7 SPL detection queries and the construction of a four-panel SOC dashboard.

Dataset Overview:

The dataset used in our project is the OpenSSH authentication log from the LogHub repository, a publicly available collection of system logs from real production environments. The log covers authentication activity on a Linux host named LabSZ and spans several weeks of continuousSSH exposure to the public internet.

The log is formatted as standard Linux syslog output: each entry begins with a timestamp, the hostname, the process name and process ID in brackets, and then the event message.

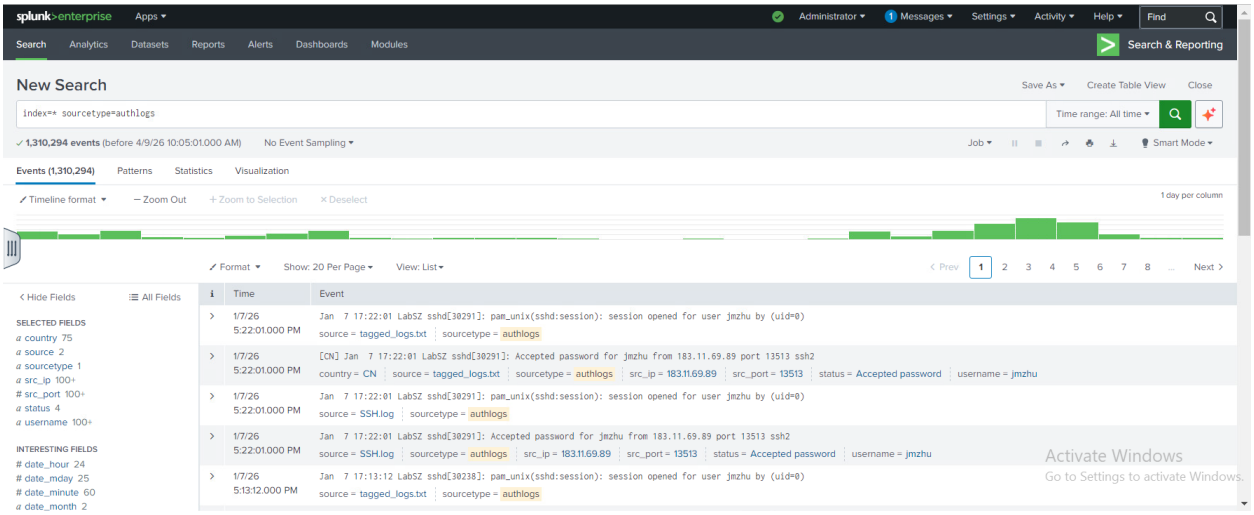
Dec 10 09:32:20 LabSZ sshd[24680]: Accepted password for fztu from 119.137.62.142 port 49116 ssh2

A Python library called geoip2 to link the logs to a local map of the internet. Instead of checking every address online, which would have taken forever, the script handles everything locally. This allows it to identify the location of every IP address in the file almost instantly. The script is designed to read the logs one line at a time to keep the process fast and prevent the computer from slowing down under the weight of such a large file. As it scans, it automatically attaches a country tag like [US] or [CN] to the start of every entry. This effectively turns a massive and messy pile of data into a clear and organized list. It makes it easy to spot exactly which countries the login attempts are coming from and which ones need to be blocked.

[CN] Dec 11 19:11:30 LabSZ sshd[29506]: Accepted password for jmzhu from 222.125.40.98 port 55185 ssh2

```python
import re
import geoip2.database
import os
DB_FILE = 'GeoLite2-Country.mmdb'
INPUT_LOG = 'ssh_logs.txt'  #filename
OUTPUT_LOG = 'tagged_logs.txt' #newly created filename

def process():
    # 1.Check if files exist
    if not os.path.exists(DB_FILE):
        print(f"Error: {DB_FILE} not found in this folder!")
        return
    if not os.path.exists(INPUT_LOG):
        print(f"Error: {INPUT_LOG} not found!")
        return
    print("Starting...")

    #Opens the database and files
    try:
        with geoip2.database.Reader(DB_FILE) as reader:
            with open(INPUT_LOG, 'r', encoding='utf-8', errors='ignore') as infile:
                with open(OUTPUT_LOG, 'w', encoding='utf-8') as outfile:

                    count = 0
                    for line in infile:
                        #Finds the IP address using Regex
                        match = re.search(r'(\d{1,3}\.){3}\d{1,3}', line)

                        if match:
                            ip = match.group(0)
                            try:
                                #Looks up the country
                                response = reader.country(ip)
                                country = response.country.iso_code
                                #Writes the country abbreviation
                                outfile.write(f"[{country}] {line}")
                            except:
                                outfile.write(f"[??] {line}")
                        else:
                            outfile.write(line)

                        count += 1
                        if count % 10000 == 0:
                            print(f"Processed {count} lines")

        print(f"Completed. Processed {count} lines.")
    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    process()
```

The key fields present in these events are the source IP address of the connection attempt, the username being targeted, the authentication result (Failed password, Accepted password, Invalid user, or Disconnecting), the country and the source port. These fields do not exist as structured columns in the raw log and must be extracted using regular expressions within Splunk.



Splunk data upload confirmation showing the authlogs sourcetype successfully configured

Detection Logic and SPL Queries:

The detection strategy for this investigation is built on a core observation: Legitimate users who mistype their password will fail to authenticate a maximum of 5 times, before either succeeding or giving up. Automated attack tools, by contrast are capable of attempting hundreds or thousands of login combinations per minute and will generate failure counts order od magnitude higher than any reasonable and plausible normal behavior.

Seven SPL queries are developed, each targeting a different dimension of the brute-force threst.

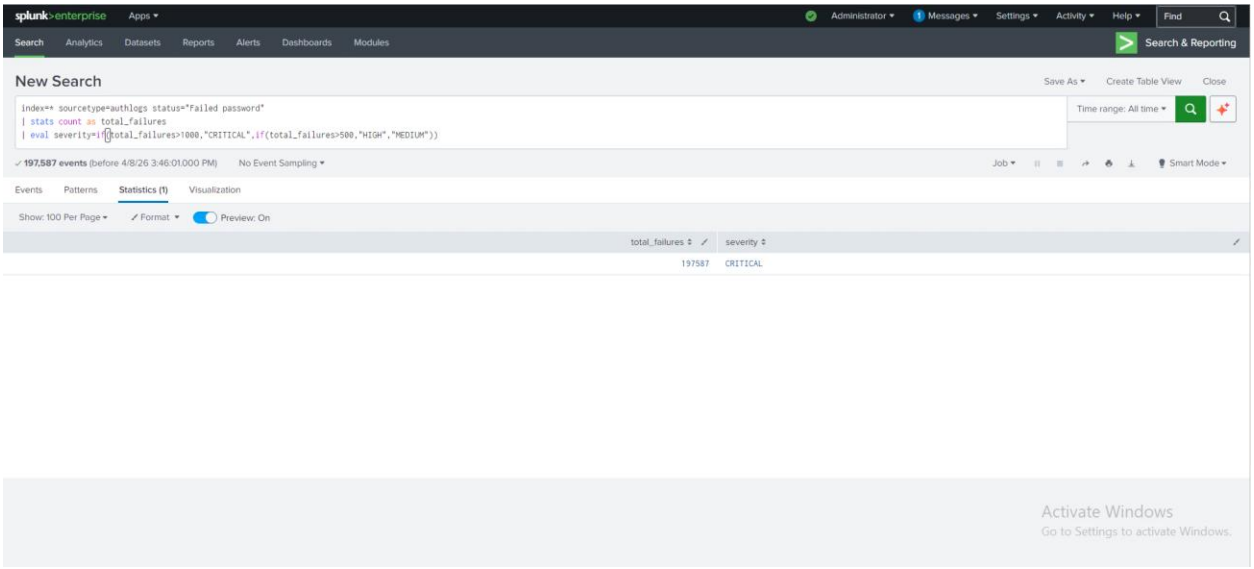
Query-1: Total Failed Login Events Overview:

The first query establishes a baseline count of all failed login events in the dataset and assigns a severity rating based on volume thresholds.

```
index=* sourcetype=authlogs status="Failed password"
| stats count as total_failures
| eval severity=if(total_failures>1000,"CRITICAL",if(total_failures>500,"HIGH","MEDIUM"))
```

Explanation:

- index=* sourcetype=authlogs - Search all indexed data where the source type is authlogs.
- status="Failed password" - Further filter to only events where the extracted status field equals 'Failed password'.
- | stats count as total_failures - The pipe (|) passes results to the stats command. count counts matching events and names the result total_failures.
- | eval severity=if(...) - The eval command creates a new field called severity using an if/else chain. If total_failures exceeds 1000 it is CRITICAL, over 500 is HIGH, otherwise MEDIUM.



Query 1 result showing total failed login count with CRITICAL severity classification

Query-2: Top Attacking IP Addresses:

The second query identifies the ten source IP addresses responsible for the greatest number of failed login attempts and enriches the results with threat intelligence from the IP blocklist lookup table.

```
index=* sourcetype=authlogs status="Failed password"
| stats count as failed_attempts by src_ip
```

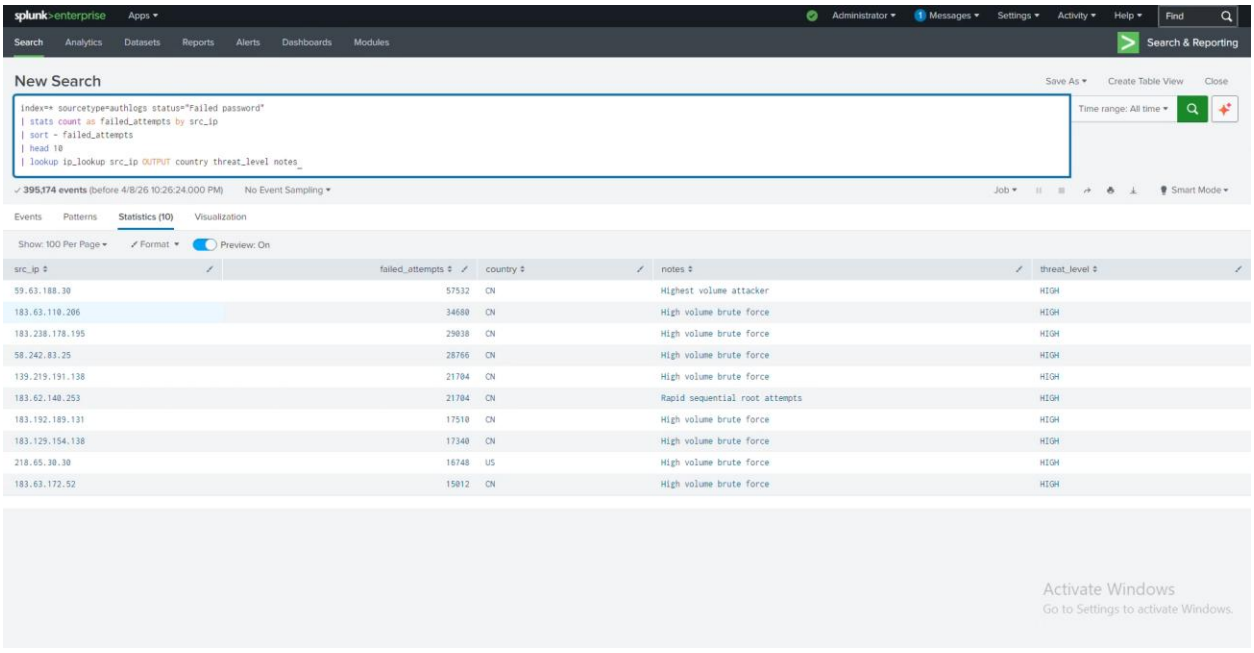
| sort - failed_attempts

| head 10

| lookup ip_lookup src_ip OUTPUT country threat_level notes

Explanation:

- | stats count as failed_attempts by src_ip - Counts failed events grouped by each unique source IP. The result is a table: one row per IP with its failure count.
- | sort - failed_attempts - Sorts the results by failed_attempts in descending order (the minus sign means descending). The IPs with the most failures appear first.
- | head 10 - Limits results to the top 10 rows. Useful for dashboards and avoiding clutter.
- | lookup ip_lookup src_ip OUTPUT country, threat_level notes - Joins ip_blocklist.csv to the results on the src_ip field. Any IP that matches gets its country and threat_level and notes columns added.



Query 2 result showing top 10 attacking IPs with country, threat level, and notes from the blocklist lookup

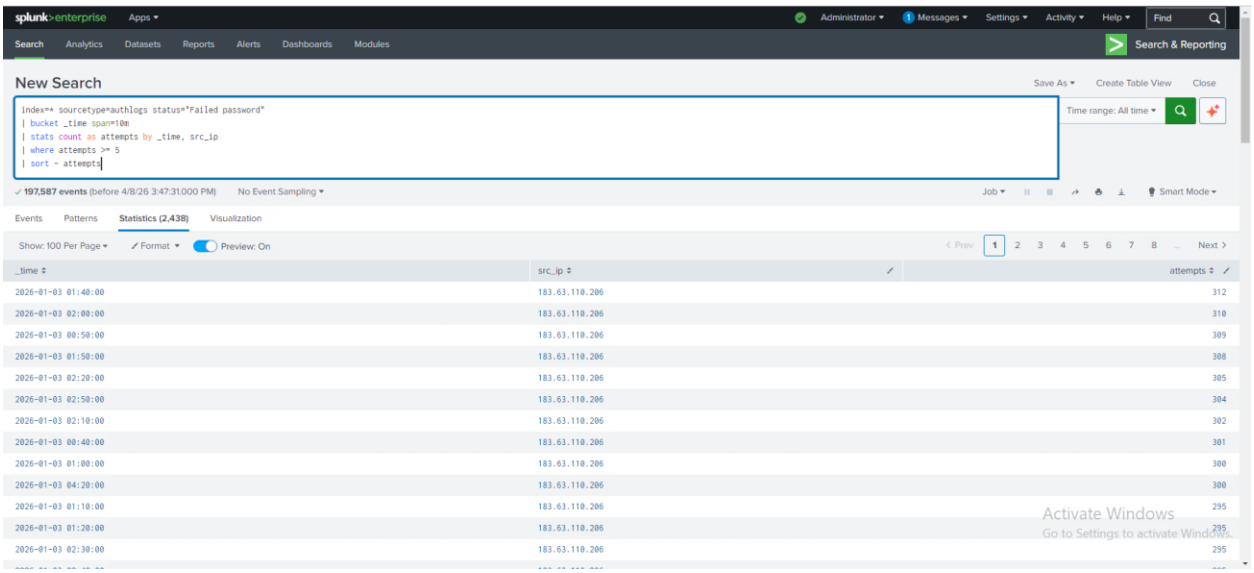
Query-3: Threshold-Based Brute Force Detection:

The third query implements a time-windowed detection rule: any source IP that generates five or more failed logins within any ten-minute window is flagged as a confirmed brute-force source.

```
index=* sourcetype=authlogs status="Failed password"
| bucket _time span=10m
| stats count as attempts by _time, src_ip
| where attempts >= 5
| sort – attempts
```

Explanation:

- | bucket _time span=10m - Groups the _time field into 10-minute buckets. Instead of individual timestamps, events are grouped into blocks such as 07:00-07:10, 07:10-07:20, etc.
- | stats count as attempts by _time, src_ip - Counts events for each combination of time bucket and source IP. This shows how many failures each IP had in each 10-minute window.
- | where attempts >= 5 - Filters results to only show IP/time combinations where failures reach or exceed 5.
- | sort - attempts - Shows the worst offenders first.



Query 3 result showing IP addresses exceeding the 5-failure threshold within 10-minute windows

Query-4: Targeted Account Analysis:

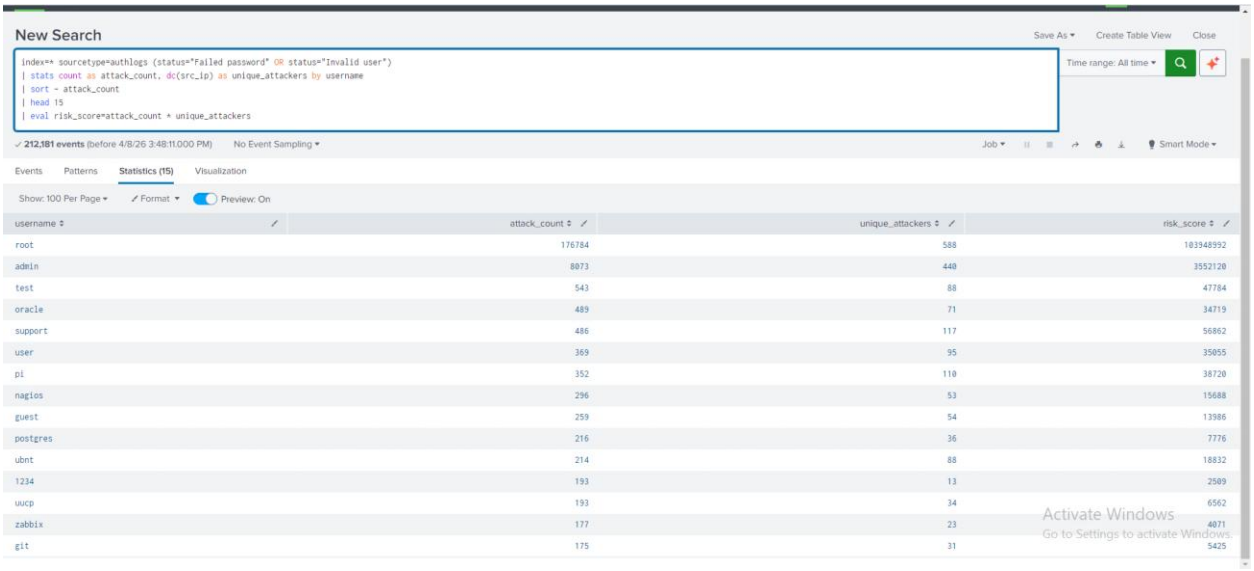
The fourth query shifts focus from attackers to victims, identifying which user accounts are being targeted most aggressively and from how many distinct source IPs.

```
index=* sourcetype=authlogs (status="Failed password" OR status="Invalid user")
| stats count as attack_count, dc(src_ip) as unique_attackers by username
| sort - attack_count
| head 15
| eval risk_score=attack_count * unique_attackers
```

Explanation:

- (status="Failed password" OR status="Invalid user") - Uses Boolean OR to include both event types. 'Invalid user' events occur when the username does not exist at all on the system, which is also a brute-force indicator.

- | stats count as attack_count, dc(src_ip) as unique_attackers by username - Two stats in one command. count gives total events; dc() is distinct count - the number of unique IP addresses that attacked each username.
- | eval risk_score=attack_count * unique_attackers - Creates a composite risk score. An account attacked 100 times from 5 IPs scores 500; one attacked 50 times from 50 IPs (distributed) scores 2500 — appropriately flagged as higher risk.



Query 4 result showing most targeted usernames with attack count, unique attackers, and composite risk score

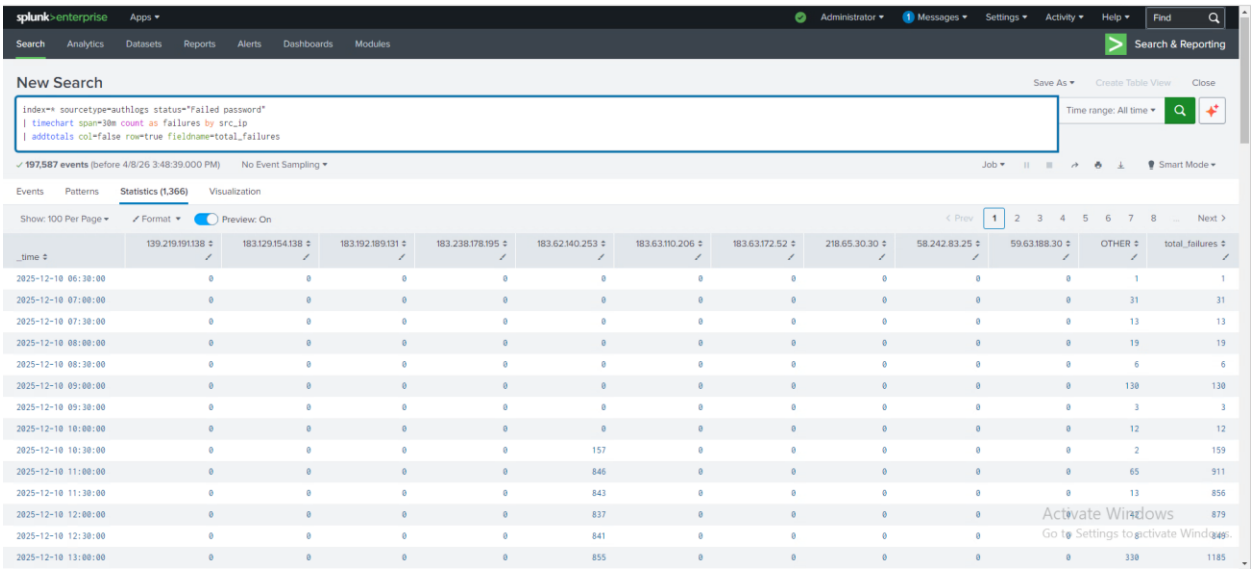
Query-5: Attack Timeline:

The fifth query produces the primary time-series visualisation for the executive dashboard panel, showing how failed login volume evolved over the observation period.

```
index=* sourcetype=authlogs status="Failed password"
| timechart span=30m count as failures by src_ip
| addtotals col=false row=true fieldname=total_failures
```

Explanation:

- | timechart span=30m count as failures by src_ip - timechart is a specialized command that creates time-series data. span=30m groups events into 30-minute intervals. count as failures names the metric. by src_ip splits the chart into one series per IP address.
- | addtotals col=false row=true fieldname=total_failures - Adds a row-total column (total_failures) summing all IP failure counts per time bucket. col=false prevents a column total; row=true adds the row total.



Query 5 result showing failed login volume over time, with individual IP series and total failures line

Query-6: Compromise Detection:

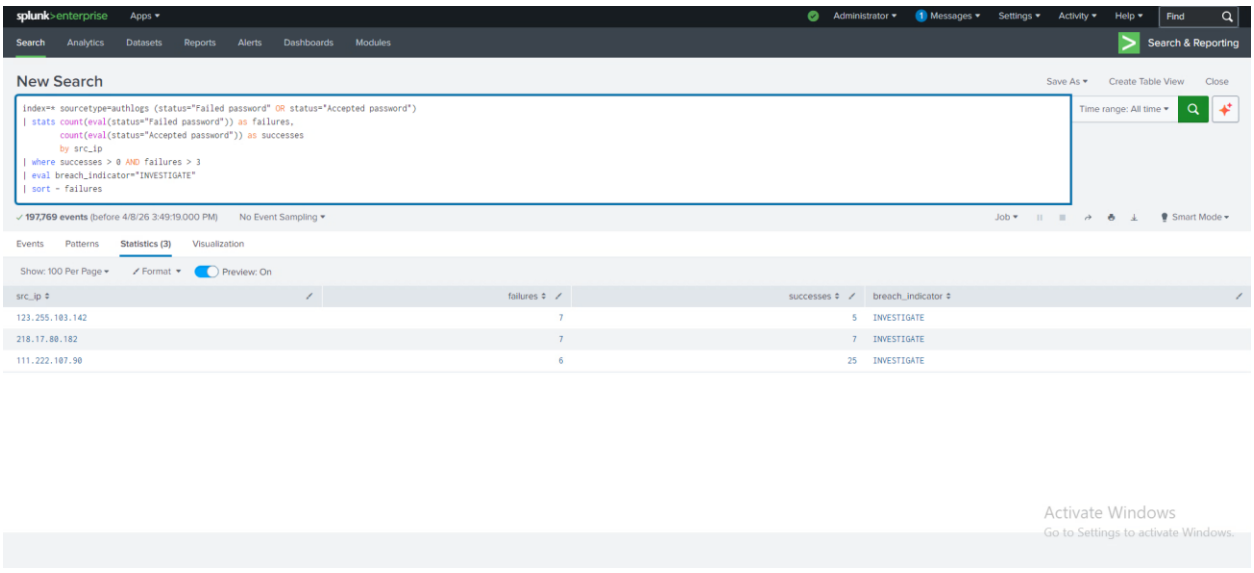
The sixth query is the most operationally significant: it identifies IP addresses that first generated multiple failed logins and then achieved at least one successful authentication - the definitive signature of a successful brute-force attack.

```
index=* sourcetype=authlogs (status="Failed password" OR status="Accepted password")
| stats count(eval(status="Failed password")) as failures,
    count(eval(status="Accepted password")) as successes
    by src_ip
```

```
| where successes > 0 AND failures > 3
| eval breach_indicator="INVESTIGATE"
| sort - failures
```

Explanation:

- count(eval(status="Failed password")) as failures - Uses eval inside stats to conditionally count. Only events where status equals 'Failed password' are counted, naming the result failures.
- count(eval(status="Accepted password")) as successes - Same pattern for successful logins. This gives you both metrics per IP in a single pass.
- | where successes > 0 AND failures > 3 - The critical filter. We only care about IPs that succeeded at least once AND had more than 3 prior failures. This combination is the brute-force success signature.
- | eval breach_indicator="INVESTIGATE" - Tags every result with a constant string for visual clarity in the dashboard table.



Query 6 result identifying IP addresses with both failed and successful logins - breach indicators

Query-7: Global Threat Origin Analysis:

The seventh query aggregates failed login attempts by country of origin, enabling geographic attribution of the attack traffic.

```
index=* sourcetype=authlogs status="Failed password"
| stats count as failed_attempts by country
| sort - failed_attempts
```

Explanation:

- index=* sourcetype=authlogs status="Failed password"

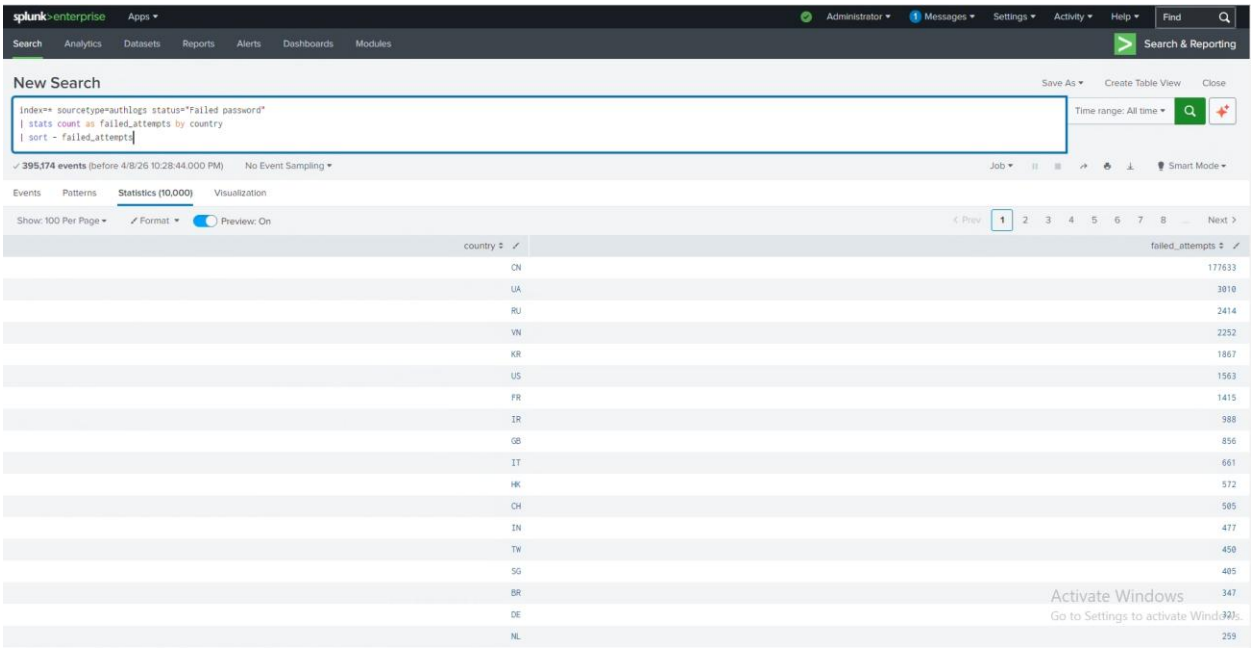
  Same base filter as all other queries - only failed password events.

- | stats count as failed_attempts by country

  Groups the failure count by the country field.

- | sort - failed_attempts

  Orders results from the country with the most attacks to the least.



Query 7 result showing failed login volume aggregated by country of origin

False Positive Mitigation:

All detection rules were set above levels that would capture legitimate user behavior. A single failed login is not an indicator of attack; five failures within ten minutes is. A successful login after one failed attempt is not suspicious; a successful login after four or more failures from the same IP is.

In production SOC environment, additional context would be used to further reduce false positives, the lookups are used to identify whether the source IP belongs to a known internal system, time-of-day analysis to flag the time and correlation with endpoint logs from target host to confirm whether the logged-in session performed any suspicious actions.

Dashboard Design:

The Brute Force Detection Dashboard is built in Splunk as a Classic with four panels.

Panel-1: Brute Force Attack Timeline:

The first panel presents the time-series line chart produced by Query 5. It is positioned at the top of the dashboard and spans the full width, providing an immediate visual overview of attack activity across the entire observation period. Spikes in the chart are visually prominent and require no technical knowledge to interpret: a large spike means many attacks occurred in a short period. This panel answers the executive question “When were we under attack and is it still happening?” without requiring the viewer to read a single number.

Panel-2: Top 10 Source IPs by Failed Logins:

The second panel displays the results of Query 2 as a statistics table, showing each of the top ten attacking IPs alongside their failed attempt count, country of origin, threat level from the blocklist, and analyst notes. This panel is the primary operational tool for an analyst responding to an active attack: it immediately identifies which IPs to block, prioritized by volume, and provides context about whether those IPs are known threats.

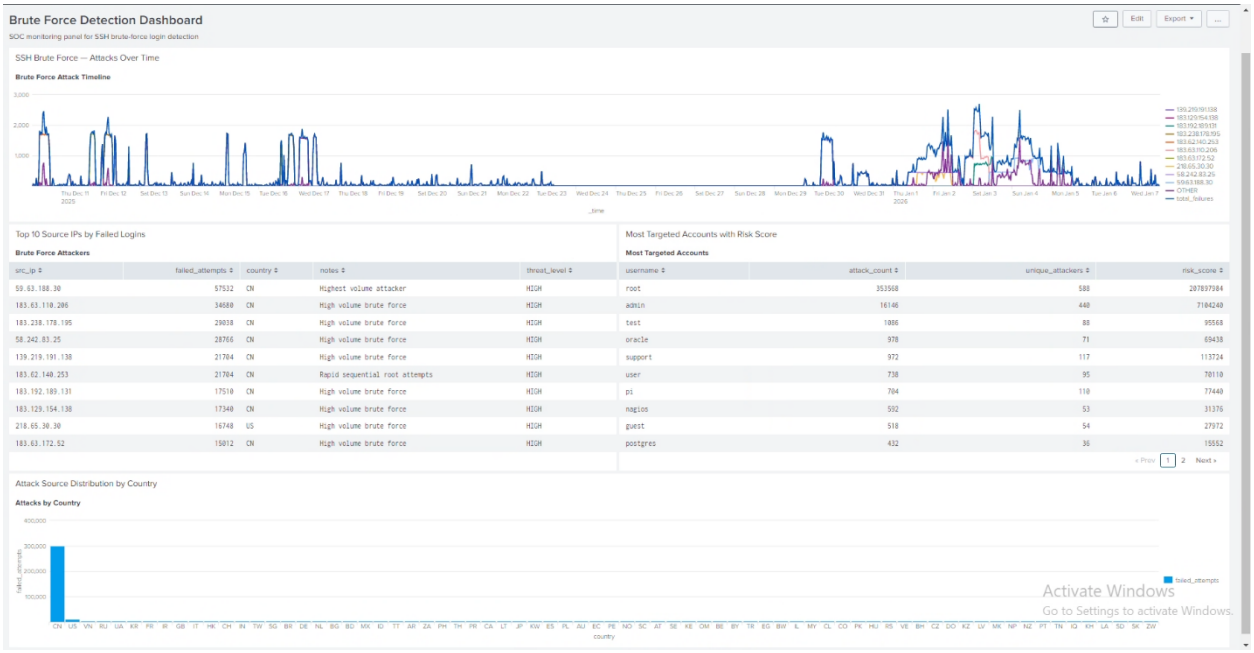
Panel-3: Most Targeted Accounts with Risk Score:

The third panel presents the results of Query 4 as a statistics table, showing the most targeted usernames alongside their attack count, the number of distinct attackers, and the composite risk score. This panel drives account-hardening decisions: accounts with high risk scores should be prioritized for password resets, multi-factor authentication enforcement, or temporary lockout depending on whether they are valid system accounts or non-existent accounts being probed.

Panel-4: Attack Source Distribution by Country:

The fourth panel visualises the results of Query 7 as a bar chart, showing failed login volume grouped by country of origin. This panel contextualises the attack traffic geographically and supports geo-blocking decisions. If a single country accounts for a disproportionate share of attack traffic and has no legitimate business relationship with the organisation, blocking that country at the firewall or load balancer level may be a cost-effective risk reduction measure.

DashBoard:



Completed Brute Force Detection Dashboard showing all four panels

REFLECTION:

Splunk's Field Extractor tool significantly simplified the field extraction process. By highlighting values in sample events, the tool generated regular expressions automatically that were accurate for the majority of event types without manual regex authoring. The lookup table pattern is directly transferable to production environments where commercial threat intelligence feeds such as AbuseIPDB or VirusTotal replace the static CSV.

The use of the pre-tagged log file for geolocation was an elegant solution that avoided the need for real-time API calls or complex enrichment pipelines. By embedding the country code directly in the log line, the country field could be extracted with a single rex command and used immediately in any query or visualisation.

Challenges Encountered:

The primary technical challenge was the inconsistent format of auth.log. Not all event types contain all fields: connection-closed events have no username, session-opened events are duplicated across PAM and sshd processes, and some failed login events use slightly different message formats depending on whether the username exists on the system. Field extractions had to be designed as optional matches rather than strict patterns to avoid generating errors on partial events.

The solution here was to experiment with the regex, that took a while but in the end it was worth it.